SOFTWARE REQUIREMENTS SPECIFICATION

TransitBD

Mass Transport Ticketing System
with Intelligent Transit Analytics

Version 1.0 · Approved

Prepared by **Group-12**
Course Project — Software Engineering
3 August 2026

Repository: github.com/Munem3/transit-ticketing-system

Revision History

Name	Date	Reason for Changes	Version
Group-12	2026-08-03	Initial SRS covering the full TransitBD system.	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for **TransitBD**, version 1.0 — a web-based mass transport ticketing platform with AI transit analytics for Bangladesh. It covers the complete system: commuter ticketing across bus, intercity train, and metro; a mock digital wallet; QR ticketing; three AI features; and an administrative control panel.

1.2 Document Conventions

- Functional requirements are tagged `REQ-n` and are uniquely numbered per feature.
- Each feature is assigned a priority of **High**, **Medium**, or **Low**.
- Priorities assigned to a feature are inherited by its detailed requirements unless stated otherwise.
- Terms in `monospace` refer to code entities, API routes, or database fields.

1.3 Intended Audience and Reading Suggestions

This document is intended for the development team, project supervisor/evaluators, testers, and future maintainers. Readers new to the project should read Section 1 (Introduction) and Section 2 (Overall Description) first, then Section 4 (System Features) for functional detail, and Section 5 for quality attributes. Evaluators may focus on Sections 2, 4, and Appendix B (Analysis Models).

1.4 Product Scope

TransitBD unifies ticketing across Dhaka Metro, intercity trains, and buses into one platform, giving commuters real-time seat availability, transparent fares, and AI-assisted journey planning, while giving transport authorities demand analytics and operational control. Objectives: reduce peak-hour friction, provide fare transparency, and enable data-driven fleet decisions. The system is a self-contained web application; payments and AI are simulated for this educational release.

1.5 References

- Project brief — "Mass Transport Ticketing System with Intelligent Transit Analytics", Group-12.
- UML analysis models — `docs/UML-DIAGRAMS.md` (use case, sequence, communication, activity, class).
- Detailed use case description — `docs/use-case-description.md`.
- Next.js, Prisma, NextAuth.js, and Google Gemini API official documentation.

- Source repository — https://github.com/Munem3/transit-ticketing-system

2. Overall Description

2.1 Product Perspective

TransitBD is a new, self-contained product (not a follow-on or replacement). It is a full-stack Next.js application in which the frontend, backend, and API layers share one codebase. The system integrates two external actors: the **Google Gemini** LLM (for AI features, with a heuristic fallback) and a **mock digital wallet** (bKash / Rocket / Card) that simulates payment. Major components:

- **Web UI** (React/Next.js, Tailwind) — commuter and admin interfaces.
- **Application server** — Next.js Server Actions & API Route Handlers.
- **Database** — Prisma ORM over SQLite (dev) / PostgreSQL (prod).
- **External services** — Gemini API; simulated wallet.

2.2 Product Functions

- Register and authenticate securely.
- Browse trips by mode with real-time seat/compartment availability.
- Hold seats with an automatic 5-minute countdown, then pay from a wallet.
- Issue instant QR-coded tickets; cancel with automatic refund.
- AI peak-demand & fare prediction, multi-modal route planning, and a support chatbot.
- Administer schedules, fare grid, and the transaction ledger.

2.3 User Classes and Characteristics

User class	Characteristics & privileges
Commuter (primary)	Registered traveller; books, pays, manages tickets, uses AI assistant. Largest, most important class.
Administrator	Operator/staff; manages trips, fares, and views the ledger & analytics. Elevated privileges.
Guest	Unauthenticated visitor; can view the landing page and register/log in.
Gemini AI (external system)	Supporting actor invoked by the AI features; not a human user.

2.4 Operating Environment

- **Client:** any modern desktop or mobile web browser (Chrome, Edge, Firefox, Safari).
- **Server:** Node.js 18+ runtime; deployable on Vercel or any Node host.

- **Database:** SQLite for development; PostgreSQL/MySQL for production.
- **Network:** HTTPS; outbound access to the Gemini API when AI is enabled.

2.5 Design and Implementation Constraints

- Implemented in **TypeScript** end-to-end using **Next.js 14 (App Router)**.
- Data access via **Prisma ORM**; authentication via **NextAuth.js** (JWT) with bcrypt hashing.
- AI features use the **Google Gemini API**; must degrade gracefully to heuristics without a key.
- Payments are **simulated** (no real gateway) for this release.
- SQLite does not support Prisma enums, so enumerated fields are stored as strings with TypeScript unions.

2.6 User Documentation

- `README.md` — setup and run instructions.
- In-app guidance (labels, demo credentials, contextual messages).
- This SRS and the UML analysis models.

2.7 Assumptions and Dependencies

- Users have internet access and a modern browser.
- The Gemini API is reachable when configured; otherwise heuristic fallbacks are used.
- Wallet balances are demo values; no real financial transactions occur.
- Seed data (routes, trips, seats) is present for demonstration.

3. External Interface Requirements

3.1 User Interfaces

Responsive, mobile-first web pages with a consistent top navigation bar (Book, My Tickets, Wallet, AI Assistant, and Admin for admins). Key screens: landing, register/login, dashboard, trip browser, interactive seat map with countdown, wallet + ledger, QR ticket, AI assistant (3 tabs), and admin control panel. Errors are shown inline with clear messages (e.g., "seat just taken", "insufficient balance").

3.2 Hardware Interfaces

No special hardware is required. The system runs on standard client devices. Conceptually, a gate-side QR scanner could read a ticket's QR code, but scanning hardware is out of scope for this release.

3.3 Software Interfaces

Interface	Purpose
Prisma ORM → SQLite/PostgreSQL	Persist users, routes, trips, seats, bookings, transactions.
NextAuth.js	Credential authentication; issues JWT session tokens.
Google Gemini API	Natural-language generation for the AI features (JSON & text).
`qrcode` library	Generate QR-code data URLs for confirmed tickets.

3.4 Communications Interfaces

- All traffic over **HTTPS**.
- Client–server data exchanged as **JSON** over HTTP (API routes) and via Server Actions.
- Sessions carried by signed **JWT**; passwords never transmitted or stored in plaintext.

4. System Features

4.1 Authentication & Account Management

4.1.1 Description and Priority

Priority: High. Secure registration and login; each account has a role and a wallet balance.

4.1.2 Stimulus/Response

User submits registration/login form → system validates, hashes/verifies password, creates a session, and redirects to the dashboard.

4.1.3 Functional Requirements

- **REQ-1:** The system shall let a visitor register with name, email, and password (min 6 chars); duplicate emails are rejected.
- **REQ-2:** The system shall store passwords using bcrypt hashing only.
- **REQ-3:** The system shall authenticate users via NextAuth credentials and issue a JWT carrying user id and role.
- **REQ-4:** New accounts shall receive a demo wallet balance recorded as an opening transaction.

4.2 Trip Search & Real-time Seat Booking

4.2.1 Description and Priority

Priority: High. Browse trips by mode and reserve seats with a hold timer.

4.2.2 Stimulus/Response

User filters trips → system lists availability and fares; user selects seats and holds them → system creates a pending booking with a countdown.

4.2.3 Functional Requirements

- **REQ-5:** The system shall list active, upcoming trips filtered by mode (bus/train/metro) with live seat counts and load.
- **REQ-6:** The system shall let a user select up to 6 available seats and place a transient hold.
- **REQ-7:** Held seats shall expire after 5 minutes and revert to available if not paid.
- **REQ-8:** The system shall prevent double-booking via a database constraint and a transactional availability re-check.

4.3 Wallet & Payments

4.3.1 Description and Priority

Priority: High. Mock wallet (bKash/Rocket/Card) with balance and ledger.

4.3.2 Functional Requirements

- **REQ-9:** The system shall let a user top up the wallet via bKash, Rocket, or Card (simulated) and record the transaction.
- **REQ-10:** On payment, the system shall verify `balance ≥ fare`, deduct atomically, and record a PURCHASE transaction.
- **REQ-11:** The system shall display a chronological transaction ledger per user.

4.4 QR Ticketing & Cancellation

4.4.1 Description and Priority

Priority: High. Issue QR tickets and support cancellation with refund.

4.4.2 Functional Requirements

- **REQ-12:** On confirmation, the system shall issue a QR-coded ticket encoding the booking reference, trip, and user.
- **REQ-13:** The system shall let a user cancel a booking; confirmed tickets are refunded to the wallet automatically.
- **REQ-14:** Cancellation shall release the associated seats back to available.

4.5 AI Transit Assistant

4.5.1 Description and Priority

Priority: Medium. Three AI features powered by Gemini, each with a heuristic fallback.

4.5.2 Functional Requirements

- **REQ-15:** The system shall forecast route demand from historical seat occupancy and advise off-peak alternatives and a suggested fare multiplier.
- **REQ-16:** The system shall generate a multi-modal route plan (metro/train/bus) over the real network with estimated time and fare.
- **REQ-17:** The system shall answer FAQs and booking-status queries via a conversational support bot grounded in the user's bookings.
- **REQ-18:** If the Gemini API is unavailable, the system shall return heuristic results without failing.

4.6 Administration

4.6.1 Description and Priority

Priority: Medium. Operational control panel for admins only.

4.6.2 Functional Requirements

- **REQ-19:** The system shall restrict `/admin` to users with role ADMIN (server-side enforced).
- **REQ-20:** Admins shall adjust a trip's fare multiplier and activate/pause trips.
- **REQ-21:** Admins shall view system stats (users, tickets, revenue) and the full transaction ledger.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Typical page reads (seat maps, listings) shall complete in under ~1 second on a broadband connection.
- Database queries shall use indexes on hot paths (trip/seat/booking lookups).
- AI responses depend on the external model; the UI shall remain responsive and show a loading state.

5.2 Safety Requirements

- No real financial transactions occur; payments are simulated, preventing monetary loss.
- Atomic transactions prevent inconsistent states (e.g., charged-but-not-booked).

5.3 Security Requirements

- Passwords stored only as bcrypt hashes; sessions via signed JWT.
- Server-side authorization on all mutating actions; admin actions re-check role.
- Input validated with Zod; secrets kept in environment variables, never committed.

5.4 Software Quality Attributes

Attribute	Target
Reliability	Transactional booking; no double-booking; auto-expiry of holds.
Usability	Responsive, mobile-first; clear flows and error messages.
Maintainability	TypeScript end-to-end; modular App Router structure.
Portability/Scalability	SQLite → PostgreSQL by config; stateless server suits horizontal scaling.
Availability	AI heuristic fallback keeps features working without external dependencies.

5.5 Business Rules

- A seat may be held by at most one booking at a time.
- Only ADMIN users may modify schedules, fares, or view the global ledger.
- Confirmed tickets are refundable to the wallet; pending holds simply expire/release.
- Fare = `Route.baseFare × Trip.fareMultiplier × seatCount`.

6. Other Requirements

- **Database:** relational schema of 7 entities (User, Route, Trip, Seat, Booking, Transaction, and enumerations).
- **Internationalization:** Bangla localization is a future enhancement (out of scope for v1.0).
- **Reuse:** the AI heuristic layer and Prisma schema are reusable across future modules.

Appendix A: Glossary

Term	Definition
Hold	A temporary 5-minute reservation of seats before payment.
Fare multiplier	A factor applied to base fare for peak pricing.
QR ticket	A QR-encoded payload proving a confirmed booking.
Server Action	A Next.js server-side function callable from the UI.
LLM	Large Language Model (Google Gemini) powering AI features.

Appendix B: Analysis Models

The following UML models accompany this SRS (see `docs/UML-DIAGRAMS.md`):

- Use Case Diagram (whole system)
- Sequence Diagrams — Book & Pay for Ticket; AI Multi-Modal Route Planning
- Communication Diagram — Cancel & Refund
- Activity Diagram — Book & Pay for Ticket
- Class Diagram (major classes, attributes, methods, relationships, multiplicities)

Appendix C: To Be Determined List

- TBD-1: Real payment gateway integration (post-demo).
- TBD-2: Real-time seat updates via WebSockets/polling.
- TBD-3: Bangla language localization.